TECHNICAL DOCUMENT 2: UNIT & AUTOMATION UI TESTING GUIDE

Movie Review & Finder Web Application

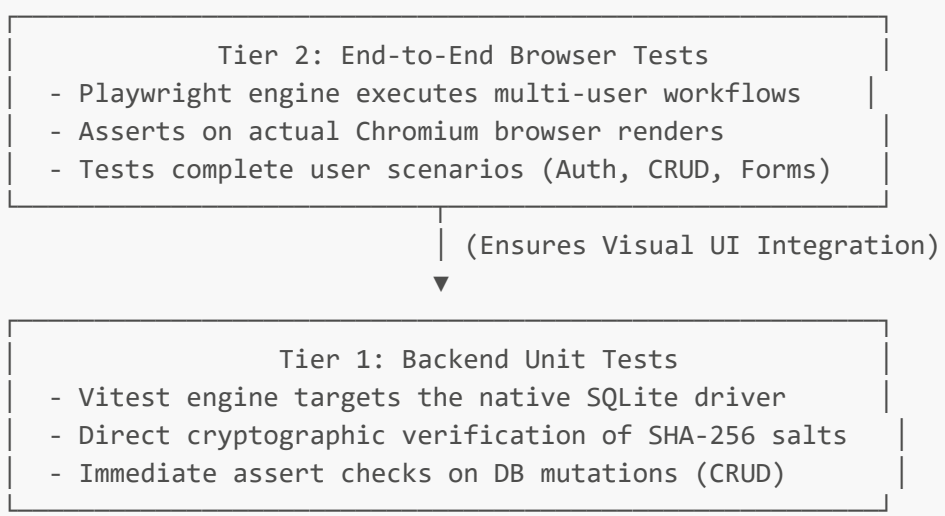
This document outlines the software testing strategy, local and pipeline test runners, unit testing configurations (`Vitest`), and End-to-End (E2E) automated browser workflows (`Playwright`). It details custom selector architecture, self-healing test patterns, and session isolation.

Table of Contents

---

1. Overview of the Testing Strategy

Our testing strategy follows a highly resilient **two-tiered approach** to guarantee absolute database integrity and zero regressions in the user experience:



- **Fast Failures**: Unit tests target independent code modules (like passwords and database CRUD) to catch logic bugs in less than a second.
- **High-Fidelity Realism**: E2E browser automation replicates manual user interactions, including clicking, filling inputs, and reading modals.

---

2. Unit Testing with Vitest

We use **Vitest** for native, lightning-fast execution of backend database queries, crypto operations, and logical functions.

Setup & Configuration

Vitest integrates seamlessly into our Vite environment. The configuration resides directly within our shared `vite.config.ts`:

```
test: {
  exclude: ['e2e/**/*', 'node_modules/**/*', 'dist/**/*'],
}
```

By explicitly excluding e2e folders from Vitest, we prevent Vitest from attempting to compile Playwright browser specs, keeping unit testing fast and lightweight.

Unit Test Cases Explained (`src/db.test.ts`)

Our test suite is organized into logical feature domains:

1. **Password Cryptography**:
   - *Goal*: Ensure password verification cannot be bypassed.
   - *Mechanism*: Invokes `hashPassword('string')` and verifies the returned format is a valid 64-character SHA-256 hexadecimal string. Asserts that hashing the same string twice produces identical signatures.
2. **User Authentication Engine**:
   - *Goal*: Validate database lookup and registration routines.
   - *Mechanism*: Registers test users in the SQLite database and runs authentications. Asserts that registering a duplicate username throws clean errors, and incorrect passwords cause authentication failure.
3. **Movie Management (CRUD)**:
   - *Goal*: Assure movies can be written, read, updated, and deleted securely.
   - *Mechanism*: Generates a dynamic movie record, writes it via `createMovie()`, asserts the generated ID is alphanumeric with a `mov_` prefix, performs a field update, reads the updated state, and finally deletes the record to verify full lifecycle safety.
4. **Activity Logging Audit**:
   - *Goal*: Validate auditable logging parameters.
   - *Mechanism*: Emits logging records via `logActivity()` containing parameters like actions, user contexts, detail strings, and mock client IPs. Confirms logs can be correctly queried and cleared.

How to Run Unit Tests

To run the unit test suite locally:

```
# Executed at the workspace root
npm run test
```

---

3. E2E Automated UI Testing with Playwright

Playwright simulates complete user actions inside headless Chromium, Firefox, or WebKit engines to test real full-stack behavior.

Setup & Configuration

Configurations are declared inside `playwright.config.ts`. It binds to our Express-Vite backend, handles server startup, and specifies output formats:

```
webServer: {
  command: 'npm run dev',
  url: 'http://localhost:3000',
  reuseExistingServer: true,
  stdout: 'ignore',
  stderr: 'pipe',
}
```

Custom Selector Architecture

To make automation scripts resilient against future visual redesigns, we implemented a **strict high-precision identifier specification**. Elements use explicit, test-specific attributes rather than fragile HTML structures.

Design Pattern:

- **Form Inputs**: Equipped with explicit, semantic `id` and corresponding `for` bindings.
- **Automation Classes**: Styled with standard visual classes alongside dedicated `.test-[name]` class indicators.
- **Actions & Buttons**: Decorated with dedicated operational IDs (e.g., `#btn-submit-auth`, `#btn-logout`).

Practical Selector Mapping Table:

UI Component	Markup Implementation	Playwright Selector
Search Input	`<input id="movie-search" ... />`	`page.locator('#movie-search')`
Login Trigger	`<button id="btn-submit-auth" ... />`	`page.locator('#btn-submit-auth')`
Logout Button	`<button id="btn-logout" ... />`	`page.locator('#btn-logout')`

UI Component	Markup Implementation	Playwright Selector
Add Movie Button	`<button id="btn-add-movie-trigger" />`	`page.locator('#btn-add-movie-trigger')`
Title Field	`<input id="movie-title" ... />`	`page.locator('#movie-title')`
Movie Modals	`<div id="movie-details-modal" ... />`	`page.locator('#movie-details-modal')`

Playwright Test Cases Explained (`e2e/auth-and-movies.spec.ts`)

Our browser spec suite evaluates 8 robust operational flows:

1. **should display the correct login header**: Verifies text formatting of the login screen to ensure assets are correctly rendered.
2. **should fail authentication on incorrect credentials**: Submits mock login payloads and asserts error banners are displayed to users.
3. **should toggle tabs**: Toggles between "Sign In" and "Create Account" panels to ensure state switches smoothly.
4. **should pre-fill sandbox accounts**: Verifies developers can use pre-fill shortcuts to instantly login and bypass manual form entries.
5. **should allow logged-in catalog actions**: Authenticates, redirects to the main catalogue interface, asserts the navigation is successful, and views profile panels.
6. **should support high-precision automation with custom IDs**: Adds a new movie via the creation modal, searches the list for the newly created movie, edits user bio data, and verifies state persistence across view switches.
7. **should add a movie, verify it exists, delete it, and assert it is removed**: Performs a complete CRUD lifecycle validation—creates a film, verifies its presence, deletes it via the UI (handling confirmation dialogs), and asserts its complete absence.
8. **should validate invalid entries on the add movie form**: Asserts form client-side constraints on rating range boundaries (e.g. max=10) and required title checks to prevent invalid database state submissions.

Session Isolation & Self-Healing Mechanics

Two features prevent automated test scripts from crashing or generating false-negative failures:

1. Session Isolation

In previous environments, test files could leak state (such as persistent login cookies) to subsequent tests, leading to race conditions. To isolate testing runtimes, we implemented cookie clearing and session checks inside the `beforeEach` block:

```
test.beforeEach(async ({ page }) => {
  await page.goto('/');

  // 1. Give components a brief window to load
  await Promise.race([
```

```
    page.locator('#btn-prefill-admin').waitFor({ state: 'visible', timeout: 5000
}).catch(() => {}),
    page.locator('#btn-logout').waitFor({ state: 'visible', timeout: 5000
}).catch(() => {})
  ]);

  // 2. Self-Healing Active Session Checker
  const logoutBtn = page.locator('#btn-logout');
  if (await logoutBtn.isVisible()) {
    await logoutBtn.click();
    await expect(page.locator('h1')).toContainText('Login for Movie Review &
Finder');
  }
});
```

2. Robust Multi-Result Match Handling

When adding dynamic items, searching for an item may yield multiple matching elements. To prevent Playwright from throwing a "strict mode violation: locator resolved to 2 elements" exception, E2E assertions target the first matching element:

```
// Target the first matching result to avoid multi-match errors
await expect(page.locator('text=The Matrix Resurrections').first()).toBeVisible();
```

How to Run E2E Tests

To run E2E browser tests locally:

1. **Ensure Chromium Web Drivers are installed**:

   ```
   npx playwright install chromium
   ```

2. **Execute the Test Suite**:

   ```
   npm run test:e2e
   ```

---

4. CI/CD Reporting & Failure Diagnostics

When running in GitHub Actions or a Docker container, Playwright provides multiple reporting options on test failures:

- **Command Line Interface**: Outputs the exact stack trace, the failed line of code, and the expected vs actual visual output.

- **Traces & Screenshots**: If a test fails, Playwright captures a full-size page screenshot saved directly inside `test-results/`.
- **Diagnostic Logs**: If an element fails to load, Playwright logs the full list of elements currently rendered in the DOM, allowing developers to quickly identify mismatched IDs.